Lab session 4: Programming Fundamentals

Problem 1: Laser game

A laser travels through a rectangular board containing empty cells and mirrors. The board contains the following characters:

- `.` represents an empty cell;
- `/` and `\` represent mirrors;
- `>`, `<`, `^` and `v` represent the laser and its initial direction.

There is exactly one laser on the board. The laser moves one cell at a time in its current direction. Empty cells do not affect the direction of the laser. When the laser reaches a mirror, its direction changes.

A `/` mirror changes the direction as follows:

laser comes from	laser continues towards
left	up
right	down
above	left
below	right

A `\` mirror changes the direction as follows:

laser comes from	laser continues towards
left	down
right	up
above	right
below	left

The laser stops when it leaves the board. You may assume that the input is such that the laser will eventually leave the board and therefore never travels in a loop. Note that the path of the laser may intersect itself. If an empty cell is visited more than once, the output must show the direction in which the laser last travels through that cell.

The first line of the input contains two positive integers n and m, giving the number of rows and columns of the board. The next n input lines each contain m characters describing the board. Your program must print the board after the complete path of the laser has been determined. Every empty cell visited by the laser must be replaced by a character indicating the direction in which the laser travels through that cell: `>`, `<`, `^`, or `v`. The mirrors themselves remain unchanged.

Example 1:

input:

```
5 5
.....
...^.
.....
.....
..\.\
```

output:

```
...^.
...^.
.....
.....
..\.\
```

Example 2:

input:

```
10 10
..>.\.\..\
/.........
.......//
\.\.......
.......//\
/.\.......
....//..//..
\./......//
.....//.//..
.\.\......
```

output:

```
..>>\.\..\
/...v.....
....v..//./
\.\.v.....
....v..//.\
/.\.v.....
<<<<//..//..
\./......//
.....//.//..
.\.\......
```

Example 3:

input:

```
6 8
........
...//.\..
........
.>.//....
..\..\..
........
```

output:

```
........
...//>\..
...^.v..
.>>//.v..
..\..\>>
........
```

Problem 2: Balance Points

Consider an $n \times m$ matrix of integers, consisting of n rows and m columns. A position (i, j) is called a *balance point* if it divides the matrix into four rectangular regions having equal sums. More precisely, (i, j) is a balance point if the following four sums are equal:

$$\sum_{r=0}^{i}\sum_{c=0}^{j} a_{r,c} = \sum_{r=0}^{i}\sum_{c=j+1}^{m-1} a_{r,c} = \sum_{r=i+1}^{n-1}\sum_{c=0}^{j} a_{r,c} = \sum_{r=i+1}^{n-1}\sum_{c=j+1}^{m-1} a_{r,c}$$

Thus, row i belongs to the two upper regions, while column j belongs to the two left regions.

The first line of the input contains two positive integers n and m, specifying the number of rows and columns of the matrix. The next n lines each contain m integers describing the matrix. Note that rows are numbered from 0 to $n - 1$ and columns from 0 to $m - 1$.

The output should be all balance points, one per line, in lexicographic order. That is, (i_1, j_1) must be printed before (i_2, j_2) if $i_1 < i_2$, or if $i_1 = i_2$ and $j_1 < j_2$. Each balance point is printed as two integers (row first, column next) separated by a space. If the matrix has no balance points, the output should be `UNBALANCED`.

Example 1:

input:

```
4 4
1 2 4 0
3 1 3 0
5 1 2 2
1 0 2 1
```

output:

```
1 1
```

Example 2:

input:

```
5 5
1 2 0 4 0
3 1 0 3 0
0 0 0 0 0
0 5 1 2 2
0 1 0 2 1
```

output:

```
1 2
2 2
```

Example 3:

input:

```
5 5
1 0 0 3 0
0 0 2 3 1
0 3 0 3 0
0 5 1 2 2
0 1 0 2 1
```

output:

```
UNBALANCED
```

Problem 3: Othello

Othello (or Reversi) is a game that is played on an 8×8 board with black and white stones. The columns are named `a` through `h`, from left to right, and the rows are numbered `1` through `8`, from top to bottom. The initial board situation is:

```
1........
2........
3........
4...WB...
5...BW...
6........
7........
8........
 abcdefgh
```

Black makes the first move, after which the players alternate between black and white. A player may place a stone only on an empty square that encloses one or more stones of the other colour in at least one direction. The possible directions are horizontal, vertical, and diagonal.

More precisely, starting from the newly placed stone and moving in one of these directions, there must first be one or more consecutive stones of the other colour, followed by a stone of the player's own colour. All stones of the other colour in such a sequence are flipped to the player's colour. A single move may enclose stones in several directions. In that case, the colour of all enclosed stones are flipped.

The input consists of a sequence of moves, separated by a space. Each move consists of a column letter followed by a row number. You may assume that every move in the input is valid.

The output must be the board position after all input moves have been played. Make sure that your output matches the format given in the following examples.

Example 1:

input:
```
d3 c3 c4
```
output:
```
1........
2........
3..WB....
4..BBB...
5...BW...
6........
7........
8........
 abcdefgh
```

Example 2:

input:
```
d3 c3 c4 f5 f6 e6 d6 c5
```
output:
```
1........
2........
3..WB....
4..WBB...
5..WWWW..
6...BBB..
7........
8........
 abcdefgh
```

Example 3:

input:
```
d3 c3 c4 f5 f6 e6 d6 c5 b4 b5 a5
```
output:
```
1........
2........
3..WB....
4.BBBB...
5BWWWWW..
6...BBB..
7........
8........
 abcdefgh
```

Problem 4: Conway's game of Takuzu

Consider an $n \times m$ board containing the characters `0`, `1`, and `.`, where `.` denotes an empty cell. The board configuration evolves in a sequence of iterations. During each iteration, the following two rules are applied both horizontally and vertically:

- If two consecutive cells contain the same value, an empty cell immediately before or after them is filled with the opposite value.
- If two equal values have exactly one empty cell between them, that empty cell is filled with the opposite value.

An iteration consists of first inspecting the entire board and determining all empty cells that can be filled using these rules. Only after the entire board has been inspected are all these cells filled simultaneously. Consequently, a value produced during an iteration cannot be used to produce another value during the same iteration; it can only be used in the next iteration. The process stops when an iteration does not produce any new values. This final board position is said to be *stable*.

The input consists of a line that contains two positive integers n and m, giving the number of rows and columns of the board. The next n lines each contain m characters, each of which is `0`, `1`, or `.`.

You may assume that the input is consistent: no empty cell can be forced to both `0` and `1` in the same iteration. The output should be the number of iterations needed to reach a position which is stable.

Example 1:

input:

```
10 10
..1.100.0.
0...1....1
1...011.10
......0..1
1.1.0011.0
1....11010
01.0..0.01
0..10.....
..0.011010
11.01....0
```

output:

```
3
```

Example 2:

input:

```
10 10
.100.10...
.1.11.1.1.
00.1.....1
10..1101.0
110...0.10
.01101...1
...1.1...0
0...1..1.0
..1.0.010.
110....0.1
```

output:

```
4
```

Example 3:

input:

```
10 10
1.........
0.....10.1
.........0
.10...0...
..........
.0........
1.....0..1
..........
........1.
0...10...0
```

output:

```
0
```

Problem 5: Reachable

Consider an $n \times m$ grid consisting of free and blocked cells. A free cell is represented by `.` and a blocked cell by `#`. Rows are numbered from 0 to $n-1$, from top to bottom.

The goal is to travel from the leftmost column to the rightmost column using only free cells. From a cell at position (r, c), the next cell must be in column $c+1$ and may be in the same row, the row immediately above, or the row immediately below. Thus, the possible next positions are

$$(r-1, c+1), \qquad (r, c+1), \qquad (r+1, c+1).$$

A move is only possible if the new position is inside the grid and contains a free cell.

The first line of the input contains two integers n and m, where $2 \leq n, m \leq 10000$, giving the number of rows and columns of the grid. The next n lines each contain exactly m characters. Each character is either `.`, representing a free cell, or `#`, representing a blocked cell.

The output of your program must the numbers of all rows for which it is possible to reach the rightmost column from the leftmost column. The row numbers must be printed in increasing order on a single line, separated by spaces. If there is no such row, the output should be `IMPOSSIBLE`.

Example 1:

input:

```
10 10
..........
..........
..........
..........
..........
..........
..........
..........
..........
..........
```

output:

```
0 1 2 3 4 5 6 7 8 9
```

Example 2:

input:

```
10 10
..........
...##.....
...##.....
...##.....
...##.....
...##.....
...##.....
...##.....
...##.....
..........
```

output:

```
0 1 2 3 6 7 8 9
```

Example 3:

input:

```
10 10
....#.....
.#..#.....
.#..#.....
.#..#.....
.#........
.#........
.#..#.....
.#..#.....
.#..#.....
....#.....
```

output:

```
IMPOSSIBLE
```